

Лабораторная работа № 1

Создание проекта Next.js с использованием React, TypeScript и App Router

1. Место работы в курсе

В течение семи лабораторных работ студент разрабатывает одно пользовательское веб-приложение. Тематика приложения выбирается студентом самостоятельно и сохраняется до завершения курса.

К седьмой работе должен быть представлен законченный проект, сопоставимый с приложением TheraCity по общей логике: приложение должно решать определённую пользовательскую задачу, иметь несколько связанных разделов, работать с предметными сущностями и поддерживать основные пользовательские сценарии.

В приложении TheraCity такими сущностями являются фотографии, маршруты и пользователи, а интерфейс включает стартовую страницу, галерею, раздел маршрутов, личный кабинет, страницы авторизации и информационный раздел. Структура проекта организована с помощью App Router и каталогов `app/start`, `app/gallery`, `app/routes`, `app/cabinet`, `app/auth` и `app/about`. В данной работе подобная структура полностью не реализуется: студент создаёт её начальный каркас.

2. Цель работы

Подготовить среду разработки и создать начальную версию веб-приложения на основе Next.js, React и TypeScript с использованием App Router.

3. Задачи работы

В ходе выполнения работы студент должен:

1. Установить Node.js и проверить его работоспособность.
2. Создать новый проект Next.js.
3. Подключить TypeScript и App Router.
4. Изучить созданную структуру каталогов и файлов.
5. Сформулировать первоначальную концепцию будущего приложения.
6. Создать главную страницу приложения.
7. Создать информационную страницу `/about`.
8. Создать типизированный React-компонент.
9. Настроить навигацию между страницами.
10. Проверить запуск и сборку проекта.

4. Теоретическая часть

4.1 React

React — библиотека для создания пользовательских интерфейсов. Интерфейс React-приложения формируется из компонентов. Компонент представляет собой JavaScript-функцию, которая возвращает разметку пользовательского интерфейса.

Согласно разделу официальной документации [Your First Component](#), компоненты позволяют разделять интерфейс на самостоятельные части и повторно использовать их в

разных местах приложения. Названия React-компонентов должны начинаться с заглавной буквы. ([React](#))

Пример React-компонента:

```
export default function ProjectTitle() {  
  return <h1>Моё веб-приложение</h1>;  
}
```

Компоненты могут включать другие компоненты:

```
function ProjectTitle() {  
  return <h1>Моё веб-приложение</h1>;  
}  
  
export default function HomePage() {  
  return (  
    <main>  
      <ProjectTitle />  
      <p>Главная страница проекта</p>  
    </main>  
  );  
}
```

JSX

Для описания разметки в React применяется JSX. В официальной документации [Writing Markup with JSX](#) JSX определяется как расширение синтаксиса JavaScript, позволяющее размещать разметку внутри JavaScript-кода. JSX похож на HTML, но является более строгим синтаксисом. Например, компонент должен возвращать один корневой элемент, а все теги должны быть закрыты. ([React](#))

```
export default function Information() {  
  const courseNumber = 3;  
  
  return (  
    <section>  
      <h2>Информация</h2>  
      <p>Курс обучения: {courseNumber}</p>  
    </section>  
  );  
}
```

Фигурные скобки используются для включения JavaScript-выражений в JSX.

Свойства компонентов

Данные передаются от родительского компонента дочернему через свойства — props. В официальном разделе [Passing Props to a Component](#) указано, что свойства передаются компонентам аналогично HTML-атрибутам и являются доступными только для чтения. ([React](#))

```

type FeatureCardProps = {
  title: string;
  description: string;
};

export default function FeatureCard({
  title,
  description,
}: FeatureCardProps) {
  return (
    <article>
      <h2>{title}</h2>
      <p>{description}</p>
    </article>
  );
}

```

В первой лабораторной работе студент должен создать хотя бы один собственный компонент и передать ему типизированные свойства.

4.2 TypeScript

TypeScript расширяет JavaScript системой типов. Корректный JavaScript-код может использоваться как TypeScript-код, однако TypeScript дополнительно анализирует типы значений и помогает обнаруживать потенциальные ошибки до выполнения программы.

В официальном руководстве [TypeScript for JavaScript Programmers](#) основным преимуществом TypeScript называется возможность выявлять неожиданное поведение программы и снижать вероятность ошибок. ([TypeScript](#))

```

let projectName: string = "Student Project";
let courseNumber: number = 3;
let isCompleted: boolean = false;

```

Если переменной типа `number` присвоить строку, TypeScript сообщит об ошибке:

```

let courseNumber: number = 3;

courseNumber = "третий";

```

Основные примитивные типы `string`, `number` и `boolean` описаны в официальном разделе [Everyday Types](#). ([TypeScript](#))

Вывод типов

TypeScript способен автоматически определять тип на основании присвоенного значения:

```

const projectName = "City Explorer";

```

В этом случае TypeScript самостоятельно определяет, что `projectName` является строкой. Механизм автоматического определения типов называется выводом типов. ([TypeScript](#))

Описание объектов

Для описания структуры данных можно использовать `type` или `interface`:

```
type ProjectFeature = {
  title: string;
  description: string;
  planned: boolean;
};
const feature: ProjectFeature = {
  title: "Каталог",
  description: "Просмотр объектов приложения",
  planned: true,
};
```

В проекте Next.js:

- `.ts` используется для TypeScript-кода без JSX;
- `.tsx` используется для TypeScript-кода, содержащего JSX;
- `tsconfig.json` содержит параметры TypeScript.

Next.js имеет встроенную поддержку TypeScript и создаёт необходимую конфигурацию при выборе TypeScript в `create-next-app`. Это описано в официальном разделе [Next.js: TypeScript](#). ([Next.js](#))

4.3 Next.js

Next.js — React-фреймворк для создания веб-приложений. React отвечает за компонентную модель пользовательского интерфейса, а Next.js добавляет структуру приложения, маршрутизацию, серверный рендеринг, обработку запросов, оптимизацию и другие средства разработки.

Официальная документация Next.js рекомендует начинать создание проекта с инструмента `create-next-app`. Он формирует структуру приложения, создаёт конфигурационные файлы и устанавливает необходимые зависимости. Отдельно устанавливать React, React DOM и TypeScript после использования `create-next-app` не требуется: генератор выполняет настройку проекта автоматически. ([Next.js](#))

Основные зависимости проекта:

```
next
react
react-dom
```

Их можно увидеть в файле `package.json`.

Next.js предоставляет стандартные команды:

```
npm run dev
npm run build
npm run start
```

- `npm run dev` запускает сервер разработки;
- `npm run build` создаёт производственную сборку;
- `npm run start` запускает ранее собранное приложение.

Назначение команд описано в официальном разделе [Installation](#). ([Next.js](#))

4.4 App Router

App Router — файловый маршрутизатор Next.js, использующий структуру каталогов приложения. Он поддерживает современные возможности React, включая Server Components, Suspense и Server Functions. ([Next.js](#))

При использовании App Router маршруты размещаются внутри каталога:

`app`

Next.js использует файловую маршрутизацию: каталоги определяют сегменты URL, а специальные файлы определяют содержимое маршрутов. Эта модель описана в официальной документации [Layouts and Pages](#). ([Next.js](#))

Файл `page.tsx`

Файл `page.tsx` определяет страницу, доступную по определённому адресу.

`app/page.tsx`

соответствует адресу:

`/`

Структура:

`app/about/page.tsx`

соответствует адресу:

`/about`

Компонент страницы должен экспортироваться по умолчанию:

```
export default function AboutPage() {
  return <h1>О проекте</h1>;
}
```

Файл `layout.tsx`

Файл `layout.tsx` определяет общую разметку для нескольких страниц. Корневой `layout` обязателен и должен содержать элементы `html` и `body`. Содержимое текущей страницы передаётся через свойство `children`. ([Next.js](#))

```
export default function RootLayout({
```

```

    children,
  }: Readonly<{
    children: React.ReactNode;
  }>) {
    return (
      <html lang="ru">
        <body>{children}</body>
      </html>
    );
  }
}

```

Server Components и Client Components

В App Router страницы и layouts по умолчанию являются Server Components. Они могут формироваться на сервере без отправки всего компонентного кода в браузер.

Client Components применяются, когда необходимы:

- обработчики событий;
- состояние;
- эффекты;
- браузерные API;
- обращение к window, localStorage и другим возможностям браузера.

Для обозначения клиентского компонента используется директива:

```
"use client";
```

Она размещается в начале файла перед импортами. В первой работе клиентские компоненты использовать не обязательно, поскольку создаётся статический каркас приложения. Подробнее различие описано в разделе [Server and Client Components](#). ([Next.js](#))

4.5 Структура проекта

Официальная документация [Project Structure](#) выделяет следующие основные каталоги и файлы:

```

student-project/
├── app/
├── public/
├── package.json
├── tsconfig.json
├── next.config.ts
└── eslint.config.mjs

```

- app — страницы, layouts и компоненты App Router;
- public — статические файлы;
- package.json — зависимости и команды проекта;
- tsconfig.json — конфигурация TypeScript;
- next.config.ts — конфигурация Next.js;
- eslint.config.mjs — конфигурация статического анализа кода.

Точный набор файлов может изменяться в зависимости от версии Next.js и параметров генератора. Каталог `src` является необязательным: Next.js позволяет размещать каталог `app` как в корне проекта, так и внутри `src`. ([Next.js](#))

В данном курсе каталог `app` размещается в корне проекта, как в проекте TheraCity.

5. Практическое задание

Этап 1. Определение концепции проекта

Выберите тему веб-приложения, которое будет разрабатываться во всех семи работах.

Необходимо сформулировать:

1. Рабочее название приложения.
2. Проблему, которую оно решает.
3. Основную целевую аудиторию.
4. Главную предметную сущность.
5. Не менее трёх предполагаемых функций.

Примеры предметных сущностей:

- фотография;
- мероприятие;
- книга;
- маршрут;
- учебный материал;
- рецепт;
- музыкальный релиз;
- музейный объект;
- задача;
- пользовательская публикация.

Студент не должен копировать тематику TheraCity. ВКР используется как пример того, как одна предметная идея превращается в связанное веб-приложение.

Этап 2. Установка Node.js

Для выполнения команд в этой и последующих лабораторных работах рекомендуется использовать Git Bash. В Windows Git Bash устанавливается вместе с Git for Windows. Также допускается использовать встроенный терминал Visual Studio Code, выбрав профиль Git Bash. В macOS и Linux можно использовать стандартное приложение «Терминал».

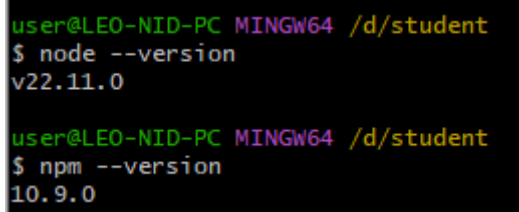
Node.js устанавливается в операционную систему с помощью установщика. Git Bash используется для выполнения команд `node`, `npm` и `pnpm` после завершения установки.

Согласно системным требованиям актуальной документации Next.js, минимальная поддерживаемая версия Node.js — 20.9. Поддерживаются Windows, Windows Subsystem for Linux, macOS и Linux. Перед установкой следует проверить актуальные требования на странице [Next.js Installation](#). ([Next.js](#))

После установки выполните:

```
node --version  
npm --version
```

Сохраните полученные номера версий для отчёта.



```
user@LEO-NID-PC MINGW64 /d/student  
$ node --version  
v22.11.0  
  
user@LEO-NID-PC MINGW64 /d/student  
$ npm --version  
10.9.0
```

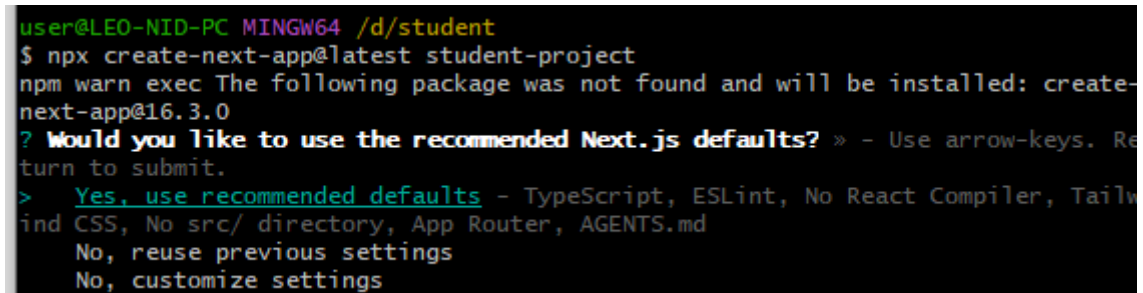
Этап 3. Создание проекта

Next.js, React, React DOM и TypeScript отдельно устанавливать не требуется. Команда create-next-app автоматически скачивает необходимые пакеты и добавляет их как локальные зависимости в папку создаваемого проекта.

Перед выполнением команды откройте Git Bash в каталоге, где должен быть создан проект. В Windows можно открыть нужную папку в Проводнике и выбрать в контекстном меню «Open Git Bash here». Команда create-next-app создаст внутри выбранного каталога новую папку с названием проекта.

Выполните команду:

```
npx create-next-app@latest student-project
```



```
user@LEO-NID-PC MINGW64 /d/student  
$ npx create-next-app@latest student-project  
npm warn exec The following package was not found and will be installed: create-  
next-app@16.3.0  
? Would you like to use the recommended Next.js defaults? » - Use arrow-keys. Re  
turn to submit.  
> Yes, use recommended defaults - TypeScript, ESLint, No React Compiler, Tailw  
ind CSS, No src/ directory, App Router, AGENTS.md  
No, reuse previous settings  
No, customize settings
```

```

defaults
Creating a new Next.js app in D:\student\student-project.

Using npm.

Initializing project with template: app-tw

Installing dependencies:
- next
- react
- react-dom

Installing devDependencies:
- @tailwindcss/postcss
- @types/node
- @types/react
- @types/react-dom
- eslint
- eslint-config-next
- tailwindcss
- typescript

```

Название student-project необходимо заменить названием собственного проекта.

При создании выберите пользовательскую настройку проекта:

```

TypeScript: Yes
Lint: ESLint
React Compiler: No
Tailwind CSS: No
src directory: No
App Router: Yes
Import alias: оставить стандартный @/*

```

Формулировки вопросов могут изменяться в новых версиях create-next-app.
Обязательными параметрами курса являются:

```

TypeScript
App Router
ESLint

```

Имя	Дата изменения	Тип	Размер
app	04.08.2026 16:51	Папка с файлами	
public	04.08.2026 16:51	Папка с файлами	
.gitignore	04.08.2026 16:51	Текстовый докум...	1 КБ
eslint.config	04.08.2026 16:51	Исходный файл J...	1 КБ
next.config	04.08.2026 16:51	Файл "TS"	1 КБ
next-env.d	04.08.2026 16:51	Файл "TS"	1 КБ
package	04.08.2026 16:51	Исходный файл J...	1 КБ
postcss.config	04.08.2026 16:51	Исходный файл J...	1 КБ
README	04.08.2026 16:51	Файл "MD"	2 КБ
tsconfig	04.08.2026 16:51	Исходный файл J...	1 КБ

Официальный перечень параметров генератора приведён в документации [create-next-app](#). ([Next.js](#))

Перейдите в каталог проекта:

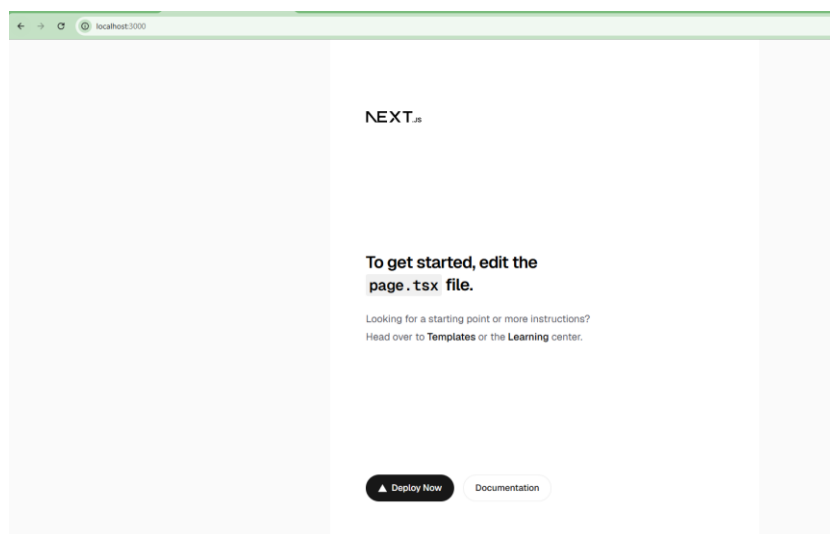
```
cd student-project
```

Запустите сервер разработки:

```
npm run dev
```

Откройте:

```
http://localhost:3000
```



Этап 4. Изучение зависимостей

Откройте файл:

```
package.json
```

Найдите зависимости:

```
"next"  
"react"  
"react-dom"
```

Найдите команды в разделе `scripts` и укажите их назначение в отчёте.

```
{ } package.json X
{} package.json > ...
1  {
2    "name": "student-project",
3    "version": "0.1.0",
4    "private": true,
5    >Debug
6    "scripts": {
7      "dev": "next dev",
8      "build": "next build",
9      "start": "next start",
10     "lint": "eslint"
11   },
12   "dependencies": {
13     "next": "16.3.0",
14     "react": "19.2.8",
15     "react-dom": "19.2.8"
16   },
17   "devDependencies": {
18     "@tailwindcss/postcss": "^4",
19     "@types/node": "^20",
20     "@types/react": "^19",
21     "@types/react-dom": "^19",
22     "eslint": "^9",
23     "eslint-config-next": "16.3.0",
24     "tailwindcss": "^4",
25     "typescript": "^5"
26   }
27 }
```

Этап 5. Создание структуры приложения

После выполнения работы проект должен иметь следующую структуру:

```
app/
├── about/
│   └── page.tsx
├── ui/
│   └── FeatureCard.tsx
├── globals.css
├── layout.tsx
└── page.tsx
```

Создание структуры проекта в Visual Studio Code

Структуру каталогов и файлов рекомендуется создавать в Visual Studio Code. Откройте корневую папку проекта через меню File → Open Folder или выполните команду code . в Git Bash, находясь в каталоге проекта.

1. В левой панели Explorer раскройте каталог app.
2. Щёлкните по каталогу app правой кнопкой мыши, выберите New Folder и введите название about.

3. Щёлкните по каталогу about правой кнопкой мыши, выберите New File и введите название page.tsx.

4. Аналогичным образом внутри каталога app создайте каталог ui.

5. Внутри каталога ui создайте файл FeatureCard.tsx.

Файлы app/globals.css, app/layout.tsx и app/page.tsx обычно создаются автоматически командой create-next-app. Повторно создавать их не требуется: в следующих этапах необходимо открыть и изменить их содержимое.

При необходимости каталоги и новые файлы можно создать из Git Bash. Команды выполняются из корневого каталога проекта, где находится файл package.json:

```
mkdir -p app/about app/ui
```

```
touch app/about/page.tsx app/ui/FeatureCard.tsx
```

После выполнения команд обновите панель Explorer в Visual Studio Code. Созданные каталоги и файлы должны появиться в структуре проекта.

Каталог ui не является специальным каталогом Next.js. В рамках курса он используется для хранения переиспользуемых компонентов интерфейса. Next.js допускает разные способы организации проектных файлов внутри каталога app. ([Next.js](#))

Этап 6. Создание компонента

Создайте файл:

```
app/ui/FeatureCard.tsx
```

Чтобы создать этот файл в Visual Studio Code, раскройте каталоги app и ui в панели Explorer, щёлкните по каталогу ui правой кнопкой мыши, выберите New File, введите FeatureCard.tsx и нажмите Enter. Если каталог ui отсутствует, сначала создайте его по инструкции этапа 5.

Альтернативный вариант для Git Bash:

```
touch app/ui/FeatureCard.tsx
```

После создания откройте файл FeatureCard.tsx в редакторе и добавьте в него код компонента.

Компонент должен:

- принимать название функции;
- принимать её краткое описание;
- использовать типизированные свойства;
- возвращать JSX-разметку.

Пример типа свойств:

```
type FeatureCardProps = {  
  title: string;  
  description: string;  
};
```

Готовый компонент должен использоваться на главной странице не менее трёх раз.

Этап 7. Главная страница

Измените файл:

`app/page.tsx`

Главная страница должна содержать:

1. Рабочее название приложения.
2. Описание решаемой проблемы.
3. Описание целевой аудитории.
4. Список из трёх предполагаемых функций.
5. Компоненты `FeatureCard`.
6. Ссылку на страницу `/about`.

Для перехода между страницами необходимо использовать компонент `Link` из `next/link`, который предназначен для навигации между маршрутами `Next.js`:

```
import Link from "next/link";  
<Link href="/about">О проекте</Link>
```

Работа встроенной навигации описана в официальном разделе [Linking and Navigating \(Next.js\)](#).

Этап 8. Страница «О проекте»

Создайте файл:

`app/about/page.tsx`

Страница должна содержать:

- название проекта;
- цель разработки;
- описание будущего результата;
- основную предметную сущность;
- предполагаемые разделы приложения;
- ссылку для возврата на главную страницу.

Страница должна открываться по адресу:

<http://localhost:3000/about>

Этап 9. Проверка проекта

Проверьте работу приложения:

```
npm run dev
```

Затем выполните статическую проверку:

```
npm run lint
```

Создайте производственную сборку:

```
npm run build
```

Проект считается технически корректным, если команды завершаются без ошибок.

```
MINGW64:/d/student/student-project
user@LEO-NID-PC MINGW64 /d/student/student-project
$ npm run dev
> student-project@0.1.0 dev
> next dev
  ▲ Next.js 15.4.5
  - Local:      http://localhost:3000
  - Network:    http://192.168.0.23:3000
  ✓ Starting...
  ✓ Ready in 1832ms
^C
user@LEO-NID-PC MINGW64 /d/student/student-project
$ npm run lint
> student-project@0.1.0 lint
> eslint
user@LEO-NID-PC MINGW64 /d/student/student-project
$ npm run build
> student-project@0.1.0 build
> next build
  ▲ Next.js 15.4.5
  Creating an optimized production build ...
  ✓ Compiled successfully in 2.8s
  ✓ Linting and checking validity of types
  ✓ Collecting page data
  ✓ Generating static pages (2/2)
  ✓ Collecting build traces
  ✓ Finalizing page optimization

Route (app)                Size      First Load JS
┌ ○ /                      128 B      102 kB
└ ○ /about                 128 B      108 kB
+ First Load JS shared by all
  ○ (Static) prerendered as static content

user@LEO-NID-PC MINGW64 /d/student/student-project
$ |
```

6. Требования к результату

После выполнения работы студент должен предоставить проект, в котором:

1. Используются Next.js, React и TypeScript.
2. Используется App Router.
3. Каталог app расположен в корне проекта.
4. Главная страница доступна по адресу /.
5. Информационная страница доступна по адресу /about.
6. Создан как минимум один переиспользуемый React-компонент.
7. Свойства компонента типизированы.
8. Компонент используется не менее трёх раз.
9. Между страницами работает навигация через next/link.
10. Проект проходит команды `npm run lint` и `npm run build`.
11. В проекте отсутствуют серверная часть, база данных и авторизация — они будут добавлены в следующих работах.

7. Содержание отчёта

Отчёт должен содержать:

1. Название и цель работы.
2. Краткое описание концепции приложения.
3. Проблему и целевую аудиторию.
4. Главную предметную сущность.
5. Версии Node.js и npm.
6. Команду создания проекта.
7. Выбранные параметры `create-next-app`.
8. Скриншот структуры каталогов.
9. Код компонента `FeatureCard`.
10. Код страниц `/` и `/about`.
11. Скриншоты обеих страниц.
12. Результат выполнения `npm run build`.
13. Ответы на контрольные вопросы.
14. Вывод по работе.

8. Источники

Теоретические источники встроены непосредственно в соответствующие разделы работы. Основными источниками являются:

- [Next.js: Getting Started](#);
- [Next.js: Installation](#);
- [Next.js: App Router](#);
- [Next.js: Project Structure](#);
- [Next.js: Layouts and Pages](#);
- [Next.js: Server and Client Components](#);
- [Next.js: TypeScript](#);
- [React: Your First Component](#);
- [React: Writing Markup with JSX](#);
- [React: Passing Props to a Component](#);
- [TypeScript for JavaScript Programmers](#);
- [TypeScript: Everyday Types](#);

Код для выполнения лабораторной работы № 1

1. Итоговая структура файлов (этап 5)

Показывает итоговую структуру файлов приложения.

```
app/
├── about/
│   └── page.tsx
├── ui/
│   └── FeatureCard.tsx
├── globals.css
├── layout.tsx
└── page.tsx
```

2. Файл app/ui/FeatureCard.tsx (этап 6)

```
// Описывает переиспользуемую карточку одной возможности
приложения.

// Типизируем входные свойства компонента.
type FeatureCardProps = Readonly<{
  title: string;
  description: string;
}>;

// Объявляем основной компонент этого файла.
export default function FeatureCard({
  title,
  description,
}: FeatureCardProps) {
  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <article className="feature-card">
      <h3>{title}</h3>
      <p>{description}</p>
    </article>
  );
}
```

3. Файл app/page.tsx (этап 7)

```
// Формирует главную страницу StudySpace из смысловых разделов и
карточек.

import Link from "next/link";
import FeatureCard from "../ui/FeatureCard";
```

```

// Объявляем основной компонент этого файла.
export default function HomePage() {
  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <main>
      <header>
        <p className="eyebrow">Учебный проект</p>
        <h1>StudySpace</h1>
        <p>
          StudySpace помогает студентам хранить учебные материалы в
одном
          месте и быстрее находить нужную информацию.
        </p>
      </header>

      <section aria-labelledby="audience-title">
        <h2 id="audience-title">Целевая аудитория</h2>
        <p>
          Студенты бакалавриата, которым необходимо
систематизировать
          конспекты, ссылки и другие учебные материалы.
        </p>
      </section>

      <section aria-labelledby="features-title">
        <h2 id="features-title">Основные функции</h2>

        <div className="feature-list">
          <FeatureCard
            title="Каталог материалов"
            description="Просмотр всех добавленных учебных
материалов."
          />

          <FeatureCard
            title="Поиск"
            description="Поиск материалов по названию и учебной
дисциплине."
          />

          <FeatureCard
            title="Избранное"
            description="Сохранение наиболее важных материалов для
быстрого доступа."
          />
        </div>
      </section>

      <Link className="main-link" href="/about">
        Подробнее о проекте
      </Link>
    </main>
  );
}

```

```

    </Link>
  </main>
);
}

```

4. Файл app/about/page.tsx (этап 8)

// Выводит страницу с описанием цели, результата и разделов проекта.

```
import Link from "next/link";
```

// Объявляем основной компонент этого файла.

```
export default function AboutPage() {
```

// Возвращаем JSX-разметку, которую React выведет на странице.

```
  return (
```

```
    <main>
```

```
      <h1>0 проекте StudySpace</h1>
```

```
      <section>
```

```
        <h2>Цель разработки</h2>
```

```
        <p>
```

Создать веб-приложение для систематизации и быстрого поиска учебных материалов.

```
      </p>
```

```
    </section>
```

```
    <section>
```

```
      <h2>Будущий результат</h2>
```

```
      <p>
```

К завершению курса приложение будет содержать несколько связанных страниц, пользовательские сценарии и работу с данными.

```
    </p>
```

```
  </section>
```

```
  <section>
```

```
    <h2>Основная предметная сущность</h2>
```

```
    <p>Учебный материал.</p>
```

```
  </section>
```

```
  <section>
```

```
    <h2>Предполагаемые разделы</h2>
```

```
    <ul>
```

```
      <li>главная страница;</li>
```

```
      <li>каталог материалов;</li>
```

```
      <li>страница отдельного материала;</li>
```

```
      <li>избранное;</li>
```

```
      <li>личный кабинет.</li>
```

```
    </ul>
```

```

    </section>

    <Link className="main-link" href="/">
      Вернуться на главную страницу
    </Link>
  </main>
);
}

```

5. Файл app/layout.tsx (этап 5)

```

// Задаёт корневой layout, метаданные и общую HTML-оболочку
приложения.

import type { Metadata } from "next";
import type { ReactNode } from "react";
import "./globals.css";

// Задаём заголовок и описание страницы для Metadata API.
export const metadata: Metadata = {
  title: "StudySpace",
  description: "Учебный проект на Next.js",
};

// Типизируем дочернее содержимое корневого layout.
type RootLayoutProps = Readonly<{
  children: ReactNode;
}>;

// Объявляем основной компонент этого файла.
export default function RootLayout({ children }: RootLayoutProps) {
  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <html lang="ru">
      <body>{children}</body>
    </html>
  );
}

```

6. Файл app/globals.css (этап 5)

```

/* Определяет базовое оформление страниц, карточек, ссылок и
состояния фокуса. */

/* Включаем предсказуемый расчёт размеров элементов. */
* {
  box-sizing: border-box;
}

html {
  background: #f5f5f5;
}

```

```

}

/* Задаём базовые параметры страницы и типографики. */
body {
  margin: 0;
  color: #202020;
  font-family: Arial, Helvetica, sans-serif;
}

/* Ограничиваем ширину основного содержимого и выравниваем его по
центру. */
main {
  width: min(900px, calc(100% - 32px));
  margin: 0 auto;
  padding: 48px 0;
}

header,
section {
  margin-bottom: 32px;
}

h1 {
  margin: 0 0 16px;
  font-size: clamp(36px, 7vw, 64px);
}

h2 {
  margin-bottom: 16px;
}

p,
li {
  line-height: 1.6;
}

.eyebrow {
  margin-bottom: 8px;
  font-weight: 700;
  text-transform: uppercase;
}

/* Автоматически распределяем карточки по доступным колонкам. */
.feature-list {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
}

.feature-card {
  padding: 20px;
}

```

```

    border: 1px solid #d7d7d7;
    border-radius: 12px;
    background: #ffffff;
  }

  .feature-card h3 {
    margin-top: 0;
  }

  .main-link {
    display: inline-block;
    font-weight: 700;
    color: #134f9c;
  }

  /* Показываем заметный фокус при клавиатурной навигации. */
  .main-link:focus-visible {
    outline: 3px solid #134f9c;
    outline-offset: 4px;
  }

```

7. Команды этапа 9

Программный код для этапа 9 не создаётся. Команды выполняются в Git Bash из корневой папки проекта, где расположен package.json.

Запускает проект, а затем проверяет стиль кода и production-сборку.

```
npm run dev
```

после проверки страницы остановить сервер сочетанием Ctrl + C

После ручной проверки выполняем автоматические проверки проекта.

```
npm run lint
```

```
npm run build
```

Ожидаемый результат: сервер разработки запускается, ESLint завершается без сообщений об ошибках, а production-сборка содержит сообщение Compiled successfully.

Примечание по версиям: если в созданном package.json отсутствует сценарий lint, студент должен использовать сценарии, автоматически созданные установленной версией create-next-app, либо добавить строку "lint": "eslint" в раздел scripts. В стандартной конфигурации курса этот сценарий создаётся автоматически.

8. Что студент должен заменить

- название StudySpace;
- описание проблемы;
- целевую аудиторию;
- основную предметную сущность;
- названия и описания трёх функций;

- перечень предполагаемых разделов приложения.

После замены текста структура компонентов и маршрутов может остаться такой же. Директива "use client" в этой работе не требуется, поскольку код не использует состояние, обработчики событий и браузерные API.